`/dev/input/event0`。

7.4.8 Bulk 传输：量大、可等待、不许出错

前面反复出现的键盘，其实不是 bulk 设备：它那份 8 字节 Boot 报告走的是 interrupt-IN（ch6）。把这份报告拿来和 bulk 对照，最能划清两者的边界。bulk 的定位是“数据多、什么时候到都行、但一个字节都不能错”——它和 interrupt 一样靠 Handshake 与 DATA0/DATA1 翻转保证可靠，却**放弃了周期承诺**：不预留带宽，只在每帧里别人用完剩下的时间见缝插针地搬，一次可以推很多包，发不动时靠设备回 NAK 做背压，主机稍后重发，直到回 ACK。

表 7.7: 同一份 8 字节键盘报告，走 interrupt 与“假设走 bulk”的差别

维度	走 Interrupt（HID 的实际做法）	走 Bulk（假设）
调度承诺	每 `bInterval`（此处 10 ms）必被轮询一次	只在剩余带宽里跑，无时间承诺
单次数据	≤ mps（8 字节），为小报告量身	高速下一个包 512 字节，搬 8 字节纯属浪费
延迟	有界，按键“立刻”出现	可能长时间不服务，按键忽快忽慢
端点	interrupt-IN EP1（描述符里属性 = 3）	需要 bulk 端点，键盘根本没有

一次 bulk 事务仍是三相，只是没有 SETUP/STATUS 那套“先谈请求”的结构，纯做单向搬运：OUT 方向主机发 `OUT` token 加 Data 包，设备回 ACK（收下）、NAK（缓冲区满，稍后再试）或 STALL（端点被 halt）；IN 方向主机发 `IN` token，设备回 Data、NAK（暂无数据）或 STALL。数据翻转保证主机不会把重传包误当成新数据，短于 mps 的包标志一次 bulk 传输结束。

本项目里真正的 bulk 用户是 MT7601U 无线网卡的 bulk-IN/bulk-OUT（ch4，详见 §7.7）：收空中进来的网络帧、发待发送的报文，数据量随流量起伏、又绝不容错，正是 bulk 的教科书场景——与“小、周期、低延迟”的键盘 interrupt 恰好互补。

7.4.9 Hub 中断驱动的 USB 热插拔

键盘按键报告和 USB 拓扑变化是两种不同的事件。按键数据由键盘自己的 interrupt-IN URB 接收；热插拔则由 LAN951x Hub 的 interrupt-IN endpoint 通知主机。当前 DWC2 为 Hub 的长期 URB 固定使用 channel 7，为唯一 HID 数据 URB 使用 channel 6。Hub 空闲时会对 interrupt-IN 返回 NAK，HCD 按 `bInterval` 重新调度请求；只有端口变化位图非零时才唤醒 Hub 事件 worker，不再每隔固定时间轮询所有端口的 EP0。

整个“插入键盘 → 枚举 → 可按键”的过程，可以串成一条从端口变化到第一个按键的总线时间线（图 7.17）：开机时四个端口都空载，控制器把 ch7 挂在 Hub 的 interrupt-IN 上等待热插拔；插入 port 3 之后由端口变化中断触发 reprobe，低速键盘的 EP0 枚举全部经 Hub 的 TT 拆成 SSPLIT/CSPLIT；驱动绑定完成后，周期 interrupt-IN 固定交给 ch6，直到某个按键的 HID 报告变成用户空间的事件。

```
Hub interrupt endpoint
  -> DWC2 channel 7 DMA receives change bitmap
  -> IRQ 9 records completion and queues hub_event_work
  -> dwc2_hub_wq examines changed ports only
  -> GET_STATUS / CLEAR_FEATURE for those ports
  -> compare connected bitmap
```

硬中断只处理 host-channel 完成并排入工作项，不在 IRQ 中进行 Hub control 传输。独立的 `dwc2_hub_wq` 根据 bitmap 中标记的端口发送 `GET_STATUS`，读取连接状态和变化位，再用 `CLEAR_FEATURE` 清除已处理的 port change feature。比较新旧 connected bitmap 后，只有拓扑确实变化才启动重枚举；因此空闲 NAK 不会造成无意义的全端口扫描。

设备拔出时，类驱动和 input/evdev 的资源按逆序回收：